LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 06 – Inharitance



MUHAMMAD ‘IZZUDDIN NABIL

103132400032

PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Inheritance (pewarisan) merupakan konsep Pemrograman Berorientasi Objek yang memungkinkan suatu kelas diturunkan dari kelas lain. Kelas asal disebut superclass, sedangkan kelas turunannya disebut subclass. Dalam Java, pewarisan diterapkan menggunakan kata kunci extends dan menunjukkan hubungan “is-a” atau “merupakan jenis dari”. Konsep ini mendukung penggunaan kembali kode karena subclass dapat menggunakan atribut dan method superclass sesuai aturan akses, sekaligus menambahkan karakteristik yang lebih spesifik.

Kata kunci super digunakan untuk mengakses atribut atau method superclass yang dapat diakses dari subclass. Selain itu, super(...) digunakan dalam constructor subclass untuk memanggil constructor superclass agar bagian objek yang berasal dari kelas induk dapat diinisialisasi. Atribut private pada superclass tidak dapat diakses secara langsung oleh subclass, sehingga aksesnya dilakukan melalui method yang disediakan oleh superclass.

Overloading merupakan pendefinisian beberapa method dengan nama sama, tetapi memiliki perbedaan jumlah, tipe, atau urutan tipe parameter. Sementara itu, overriding merupakan pendefinisian ulang method superclass pada subclass dengan signature yang sama dan tipe kembalian yang sesuai. Overriding mendukung polymorphism, yaitu kemampuan objek untuk memberikan perilaku berbeda melalui pemanggilan method yang sama sesuai kelas objek sebenarnya.

B. Guided

1. Inheritance

```java
package guided.gd1;

public class A {
    int x;
    int y;

    void tampilXY() {
        System.out.println("nilai x: " + x + " nilai y: " + y);
    }
}
```

```java
package guided.gd1;

public class B extends A{
    int z;

    void jumlahXY(){
```

```
        System.out.println("Jumlah (x+y+z) = " + (x + y + z));
    }
}
```

```
package guided.gd1;

public class Demo {
    public static void main(String[] args) {
        A superclass = new A();
        B subclass = new B();

        superclass.x = 3;
        superclass.y = 4;

        System.out.println("Superclass: ");
        superclass.tampilXY();

        subclass.x = 1;
        subclass.y = 2;
        subclass.z = 5;

        System.out.println("Subclass: ");
        subclass.jumlahXY();
        subclass.tampilXY();
    }
}
```

Output :

```
PS D:\Praktikum PBO\modul-06-inheritance>  & 'C:\Users\Lenovo\.
n32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessag
Storage\df682591a43f03847165379bb9e0dbe6\redhat.java\jdt_ws\mod
Superclass:
nilai x: 3 nilai y: 4
Subclass:
Jumlah (x+y+z) = 8
nilai x: 1 nilai y: 2
```

2. Keyword Super

```
package guided.gd2;

public class Employee {
    private String name;
    String departement;
```

```
    public Employee(String name){
        this.name = name;
    }

    public void tampilNama(){
        System.out.println("nama: " + name);
    }
}
```

```
package guided.gd2;

public class Manager extends Employee{
    private String alamat;

    public Manager(String name, String alamat){
        super(name);
        this.alamat = alamat;
    }

    public void tampilInformasi(){
        super.tampilNama();
        System.out.println("alamat: " + alamat);
        System.out.println("departemen: " + super.departement);
    }
}
```

```
package guided.gd2;

public class Demo {
    public static void main(String[] args) {
        Manager adi = new Manager("Adi", "Sukabirus");
        adi.departement = "SDM";
        adi.tampilInformasi();
    }
}
```

Output :

```
PS D:\Praktikum PBO\modul-06-inheritance>  d:; cd 'd:\Praktikum
va-1.58.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX
ng\Code\User\workspaceStorage\df682591a43f03847165379bb9e0dbe6\r
nama: Adi
alamat: Sukabirus
departemen: SDM
```

3. Overloading Method

```java
package guided.gd3;

public class Lagu {
    String pencipta;
    String judul;

    void IsiParam(String param1) {
        judul = param1;
        pencipta = "Tidak dikenal";
    }

    void IsiParam(String param1, String param2) {
        judul = param1;
        pencipta = param2;
    }

    void tampilkan() {
        System.out.println("Judul : " + judul + ", pencipta : " +
pencipta);
    }
}
```

```java
package guided.gd3;

public class Demo {
    public static void main(String[] args) {
        Lagu d, e;
        d = new Lagu();
        e = new Lagu();

        d.IsiParam("Lagu 1");
        e.IsiParam("Lagu 2", "aku");

        d.tampilkan();
        e.tampilkan();
    }
}
```

Output :

```
PS D:\Praktikum PBO\modul-06-inheritance>  d:; cd 'd:\Praktikum
va-1.58.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX
ng\Code\User\workspaceStorage\df682591a43f03847165379bb9e0dbe6\r
Judul : Lagu 1, pencipta : Tidak dikenal
Judul : Lagu 2, pencipta : aku
```

4. Overriding Method

```java
package guided.gd4;

public class A {
    public void tampilkan(){
        System.out.println("Method milik class A dipanggil...");
    }
}
```

```java
package guided.gd4;

public class B extends A{
    public void tampilkan(){
        super.tampilkan();
        System.out.println("Method milik class B dipanggil...");
    }
}
```

```java
package guided.gd4;

public class Demo {
    public static void main(String[] args) {
        B  overridB = new B();
        overridB.tampilkan();
    }
}
```

Output :

```
PS D:\Praktikum PBO\modul-06-inheritance>  d:; cd 'd:\Praktikum PBO
va-1.58.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+Sh
ng\Code\User\workspaceStorage\df682591a43f03847165379bb9e0dbe6\redha
Method milik class A dipanggil...
Method milik class B dipanggil...
```

C. Unguided

1. Soal Unguided 1

Sebuah tim machine learning ingin membuat program untuk membandingkan estimasi biaya komputasi dari beberapa jenis model yang akan dilatih. Ada tiga jenis model: model regresi, model klasifikasi, dan model klasifikasi berbasis deep learning yang merupakan pengembangan dari model klasifikasi biasa.

Buatlah program Java dengan ketentuan: Penamaan kelas, atribut, dan method ditentukan sendiri oleh Anda dan semua atribut bersifat private.

Jawaban :

```
package unguided;

public class ModelML {
    private String namaModel;
    private int jumlahDataLatih;

    // Constructor
    public ModelML(String namaModel, int jumlahDataLatih) {
        this.namaModel = namaModel;
        this.jumlahDataLatih = jumlahDataLatih;
    }

    // Behavior
    public void tampilInformasi() {
        System.out.println("Model: " + this.namaModel + " | Data
latih: " + this.jumlahDataLatih);
    }

    public int hitungBiaya() {
        return this.jumlahDataLatih * 2;
    }

    public int hitungBiaya(int epoch) {
        return this.jumlahDataLatih * epoch;
    }

    // Getter
    public String getNamaModel() {
        return this.namaModel;
    }

    public int getJumlahDataLatih() {
        return this.jumlahDataLatih;
```

```
    }

    // Setter
    public void setNamaModel(String namaModel) {
        this.namaModel = namaModel;
    }

    public void setJumlahDataLatih(int jumlahDataLatih) {
        this.jumlahDataLatih = jumlahDataLatih;
    }
}
```

```
package unguided;

public class ModelKlasifikasi extends ModelML {
    private int jumlahKelas;

    // Constructor
    public ModelKlasifikasi(String namaModel, int jumlahDataLatih,
int jumlahKelas) {
        super(namaModel, jumlahDataLatih);
        this.jumlahKelas = jumlahKelas;
    }

    // Behavior
    public void tampilInformasi() {
        super.tampilInformasi();
        System.out.println("Jumlah kelas: " + this.jumlahKelas);
    }

    public int hitungBiaya() {
        return super.hitungBiaya() * this.jumlahKelas;
    }

    // Getter
    public int getJumlahKelas() {
        return this.jumlahKelas;
    }

    // Setter
    public void setJumlahKelas(int jumlahKelas) {
        this.jumlahKelas = jumlahKelas;
    }
}
```

```java
package unguided;

public class ModelRegresi extends ModelML {
    private int jumlahFitur;

    // Constructor
    public ModelRegresi(String namaModel, int jumlahDataLatih, int
jumlahFitur) {
        super(namaModel, jumlahDataLatih);
        this.jumlahFitur = jumlahFitur;
    }

    // Behavior
    public void tampilInformasi() {
        super.tampilInformasi();
        System.out.println("Jumlah fitur: " + this.jumlahFitur);
    }

    public int hitungBiaya() {
        return super.hitungBiaya() + (this.jumlahFitur * 10);
    }

    public int hitungBiaya(int epoch, int batchSize) {
        return (this.getJumlahDataLatih() * epoch) / batchSize;
    }

    // Getter
    public int getJumlahFitur() {
        return this.jumlahFitur;
    }

    // Setter
    public void setJumlahFitur(int jumlahFitur) {
        this.jumlahFitur = jumlahFitur;
    }
}
```

```java
package unguided;

public class ModelDL extends ModelKlasifikasi {
    private int jumlahLayer;
```

```
    // Constructor
    public ModelDL(String namaModel, int jumlahDataLatih, int
jumlahKelas, int jumlahLayer) {
        super(namaModel, jumlahDataLatih, jumlahKelas);
        this.jumlahLayer = jumlahLayer;
    }

    // Behavior
    public void tampilInformasi() {
        super.tampilInformasi();
        System.out.println("Jumlah layer: " + this.jumlahLayer);
    }

    public int hitungBiaya() {
        return super.hitungBiaya() + (this.jumlahLayer * 200);
    }

    // Getter
    public int getJumlahLayer() {
        return this.jumlahLayer;
    }

    // Setter
    public void setJumlahLayer(int jumlahLayer) {
        this.jumlahLayer = jumlahLayer;
    }
}
```

```
package unguided;

public class Main {
    // Behavior
    public static void main(String[] args) {
        ModelML[] daftarModel = new ModelML[3];
        daftarModel[0] = new ModelRegresi("Regresi Linier", 500, 4);
        daftarModel[1] = new ModelKlasifikasi("Decision Tree", 800,
3);
        daftarModel[2] = new ModelDL("Neural Network", 1000, 5, 3);

        int totalBiaya = 0;
        for (ModelML model : daftarModel) {
            model.tampilInformasi();
            int biaya = model.hitungBiaya();
            System.out.println("Biaya komputasi: " + biaya);
```

```
            System.out.println("---");
            totalBiaya += biaya;
        }

        System.out.println("Total biaya: " + totalBiaya);
        System.out.println("Biaya 10 epoch (Neural Network): " +
daftarModel[2].hitungBiaya(10));
        System.out.println("Biaya 10 epoch (Regresi Linier): " +
daftarModel[0].hitungBiaya(10));
        System.out.println("Biaya 10 epoch batch 50 (Regresi Linier):
" + ((ModelRegresi) daftarModel[0]).hitungBiaya(10, 50));
    }
}
```

Output :

```
PS D:\Praktikum PBO\modul-06-inheritance>  d:; cd 'd:\Praktik
va-1.58.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '
ng\Code\User\workspaceStorage\df682591a43f03847165379bb9e0dbe
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
---
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
---
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
---
Total biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100
```

Penjelasan :

Program unguided ini merupakan sistem pemodelan estimasi biaya komputasi machine learning yang mengintegrasikan konsep multilevel inheritance, enkapsulasi, method overriding, dan method overloading. Sistem diawali dengan kelas induk ModelML yang mendefinisikan atribut umum berupa nama model dan jumlah data latih beserta rumus dasar komputasi. Kemudian diperluas oleh kelas turunan ModelRegresi yang menambahkan atribut fitur serta variasi perhitungan berbasis batch size, kelas ModelKlasifikasi yang memperhitungkan jumlah kelas, dan kelas ModelDL yang mewarisi kelas klasifikasi secara bertingkat dengan menyertakan jumlah layer

jaringan. Seluruh kelas turunan memanfaatkan kata kunci super untuk memanggil konstruktor dan menimpa (override) metode perhitungan biaya sesuai karakteristik masing-masing. Pada kelas Main, ketiga objek model dikelola secara polimorfik di dalam satu array ModelML[], di mana program mengeksekusi perulangan untuk mencetak spesifikasi model, menjumlahkan total biaya, serta membuktikan fleksibilitas pemanggilan variasi method per epoch dan ukuran batch.

D. Kesimpulan

1. Pewarisan (extends): Memungkinkan kelas anak (subclass) mewarisi atribut dan method dari kelas induk (superclass), sehingga meningkatkan efisiensi dan mencegah duplikasi kode (code reusability).
2. Kata Kunci super: Digunakan untuk memanggil konstruktor kelas induk (super(...)) saat inisialisasi objek anak, serta mengakses kembali implementasi method milik kelas induk.
3. Method Overloading vs Overriding:
   - Overloading: Membuat beberapa method bernama sama dengan jumlah/tipe parameter berbeda dalam satu kelas.
   - Overriding: Menulis ulang logika method warisan pada kelas anak agar perilakunya sesuai dengan kebutuhan spesifik kelas tersebut.
4. Pewarisan Bertingkat & Polimorfisme: Mendukung hubungan hierarki berantai (multilevel inheritance seperti kelas model deep learning), serta memungkinkan berbagai objek turunan dikelola secara seragam dalam satu array bertipe kelas induk (polymorphic array).

Penerapan inheritance, super, overloading, dan overriding memberikan pondasi yang kokoh dalam merancang sistem perangkat lunak yang modular, terstruktur, mudah dikembangkan (extensible), serta mencerminkan relasi hierarki dunia nyata secara akurat.